Freight Cargo Inventory and Shipment Management System Project Progress Report 3 - Functional Design

Muhammad Ahmad 30239993

Syed Essam Uddin Khawaja 30242943

Muhammad Zohaib Talat 30257908

CPSC471 W26

Group G-4

User-End Functionalities

The system supports two end-users: Admin and Warehouse Staff. Their functionalities are described below according to the approved relational model and system workflow.

User Interaction Workflow

The system is designed around a collaborative workflow between the two user roles:

1. Admin creates and manages the foundational master data (warehouses, locations, carriers, routes, and shipments), establishing the operational environment for the system.
2. Warehouse Staff perform day-to-day operations within that environment, updating cargo inventory quantities and loading cargo items into containers assigned to shipments.
3. Admin then monitors the outcomes of staff operations through analytics and reporting screens, viewing warehouse inventory totals, container utilization, and shipment summaries derived from staff activity.

This workflow ensures a clear separation of responsibilities while demonstrating end-to-end interaction between the two user types.

Admin

Goal: Manage master data, operational data, and access control.

Authentication / Access

- Sign in using Supabase Auth (email + password).
- Access all screens (operations, master data, analytics, and admin screens).
- System determines role by matching authenticated email to users.email and checking admins.

User & Role Management

- Create or update an application user in users.
- Assign role by inserting into:
    - admins (admin user), or
    - warehouse_staff (staff user).
- Assign an admin to manage a warehouse by inserting into manages (admin_user_id, warehouse_id).

Master Data Management (CRUD)

- Warehouses
    - Create a warehouse (warehouses) with name, address, capacity, and location.
    - View the warehouse list.
    - Delete a warehouse (when allowed by constraints).
- Locations
    - Create a location (locations) with name and type.
    - View locations.
    - Delete locations (when allowed by constraints).
- Carriers

        - Create a carrier (carriers) with name and carrier type.
        - View carriers.
        - Delete carriers.
    - Routes
        - Create a route (routes) with estimated time.
        - View routes.
        - Delete routes.

Shipment & Container Operations

- Create a shipment (shipments) by providing:
    - shipment_date, status, carrier_id, route_id, origin_loc_id, destination_loc_id.
- View shipments list, including filtering (by status) and pagination.
- Assign a container to a shipment by updating containers.shipment_id.

Analytics / Reporting

- View summary reports (from SQL views and queries), including:
    - Warehouse inventory totals (units and total weight).
    - Shipment summary (container counts and cargo counts).
    - Container utilization (current cargo weight vs max capacity).
    - Warehouse capacity vs usage.

Warehouse Staff

Goal: Perform day-to-day warehouse operations (inventory + cargo handling).

Authentication / Access

- Sign in using Supabase Auth (email + password).
- Access only operational screens (limited sidebar + redirects).
- System determines role by matching authenticated email to users.email and checking warehouse_staff.

Inventory Operations

- View inventory records (inventory_records) for cargo items in warehouses.
- Adjust inventory quantity by providing:
    - cargo_id, warehouse_id, delta_quantity.
- System updates inventory via fn_update_inventory(...) and prevents negative totals.
- System logs the adjustment to audit_logs.

Cargo Handling

- Move cargo to a container by inserting into container_cargo (cargo_id, container_id).
- System checks container capacity using:
    - total cargo weight currently assigned to the container,
    - containers.max_capacity,
    - and blocks the operation if capacity would be exceeded.

Cargo Item Creation

- Create a new cargo item in cargo_items (cargo_type, weight).
- Optionally seed an initial inventory record in inventory_records for a warehouse (warehouse_id, initial quantity).

Operational Visibility (Read-only support)

- View shipments, containers, and cargo lists to support warehouse workflows.
- Warehouse Staff have read-only access to shipment, container, and cargo overview screens outside of their permitted warehouse operations, and cannot perform administrative master-data changes.

SQL DDL and DML Statements

CREATE Database

```
CREATE DATABASE freight_cargo_management;
```

CREATE Tables:

```
-- Core user and specialization tables

CREATE TABLE IF NOT EXISTS users (
  user_id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password TEXT NOT NULL,
  role_id INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS admins (
  user_id BIGINT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
  admin_level INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS warehouse_staff (
  user_id BIGINT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
  job_title TEXT NOT NULL,
  shift TEXT NOT NULL
);

-- Location and warehouse tables

CREATE TABLE IF NOT EXISTS locations (
  location_id BIGSERIAL PRIMARY KEY,
  location_name TEXT NOT NULL,
  location_type TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS warehouses (
  warehouse_id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  address TEXT NOT NULL,
  capacity INTEGER NOT NULL,
  location_id BIGINT NOT NULL REFERENCES locations(location_id) ON DELETE RESTRICT
);

-- Relationship: Admin MANAGES Warehouse

CREATE TABLE IF NOT EXISTS manages (
  admin_user_id BIGINT NOT NULL REFERENCES admins(user_id) ON DELETE CASCADE,
```

```
  warehouse_id BIGINT NOT NULL REFERENCES warehouses(warehouse_id) ON DELETE
CASCADE,
  PRIMARY KEY (admin_user_id, warehouse_id)
);

-- Cargo and inventory tables

CREATE TABLE IF NOT EXISTS cargo_items (
  cargo_id BIGSERIAL PRIMARY KEY,
  cargo_type TEXT NOT NULL,
  weight NUMERIC(12,2) NOT NULL
);

CREATE TABLE IF NOT EXISTS inventory_records (
  cargo_id BIGINT NOT NULL REFERENCES cargo_items(cargo_id) ON DELETE RESTRICT,
  warehouse_id BIGINT NOT NULL REFERENCES warehouses(warehouse_id) ON DELETE
RESTRICT,
  quantity_stored INTEGER NOT NULL,
  last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (cargo_id, warehouse_id)
);

-- Carrier and route tables

CREATE TABLE IF NOT EXISTS carriers (
  carrier_id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  carrier_type TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS routes (
  route_id BIGSERIAL PRIMARY KEY,
  estimated_time INTEGER NOT NULL
);

-- Shipment and container tables

CREATE TABLE IF NOT EXISTS shipments (
  shipment_id BIGSERIAL PRIMARY KEY,
  shipment_date DATE NOT NULL,
  status TEXT NOT NULL,
  carrier_id BIGINT NOT NULL REFERENCES carriers(carrier_id) ON DELETE RESTRICT,
  route_id BIGINT NOT NULL REFERENCES routes(route_id) ON DELETE RESTRICT,
  origin_loc_id BIGINT NOT NULL REFERENCES locations(location_id) ON DELETE
RESTRICT,
  destination_loc_id BIGINT NOT NULL REFERENCES locations(location_id) ON DELETE
RESTRICT
);
```

```
CREATE TABLE IF NOT EXISTS containers (
  container_id BIGSERIAL PRIMARY KEY,
  container_type TEXT NOT NULL,
  max_capacity NUMERIC(12,2) NOT NULL,
  shipment_id BIGINT NULL REFERENCES shipments(shipment_id) ON DELETE SET NULL
);

-- Relationship: Container CONTAINS CargoItem (through associative table)

CREATE TABLE IF NOT EXISTS container_cargo (
  cargo_id BIGINT NOT NULL REFERENCES cargo_items(cargo_id) ON DELETE CASCADE,
  container_id BIGINT NOT NULL REFERENCES containers(container_id) ON DELETE
CASCADE,
  PRIMARY KEY (cargo_id, container_id)
);
```

INSERT/UPDATE/SELECT/DELETE Tables (these are changed depending on user input):

```
-- Insert a location
INSERT INTO locations(location_name, location_type)
VALUES ('Calgary Main Warehouse', 'WAREHOUSE');

INSERT INTO locations(location_name, location_type)
VALUES ('Edmonton Distribution Hub', 'DISTRIBUTION_CENTER');

-- Insert a warehouse linked to the above location
INSERT INTO warehouses(name, address, capacity, location_id)
VALUES ('Calgary Main', '123 Logistics Ave, Calgary, AB', 5000, 1);

-- Insert users (one admin, one warehouse staff)
INSERT INTO users(name, email, password, role_id)
VALUES ('Alice Johnson', 'alice@freightco.com', 'hashed_password_1', 1);

INSERT INTO users(name, email, password, role_id)
VALUES ('Bob Smith', 'bob@freightco.com', 'hashed_password_2', 2);

-- Assign roles
INSERT INTO admins(user_id, admin_level)
VALUES (1, 1);

INSERT INTO warehouse_staff(user_id, job_title, shift)
VALUES (2, 'Inventory Coordinator', 'DAY');

-- Assign admin to manage a warehouse
INSERT INTO manages(admin_user_id, warehouse_id)
VALUES (1, 1);
```

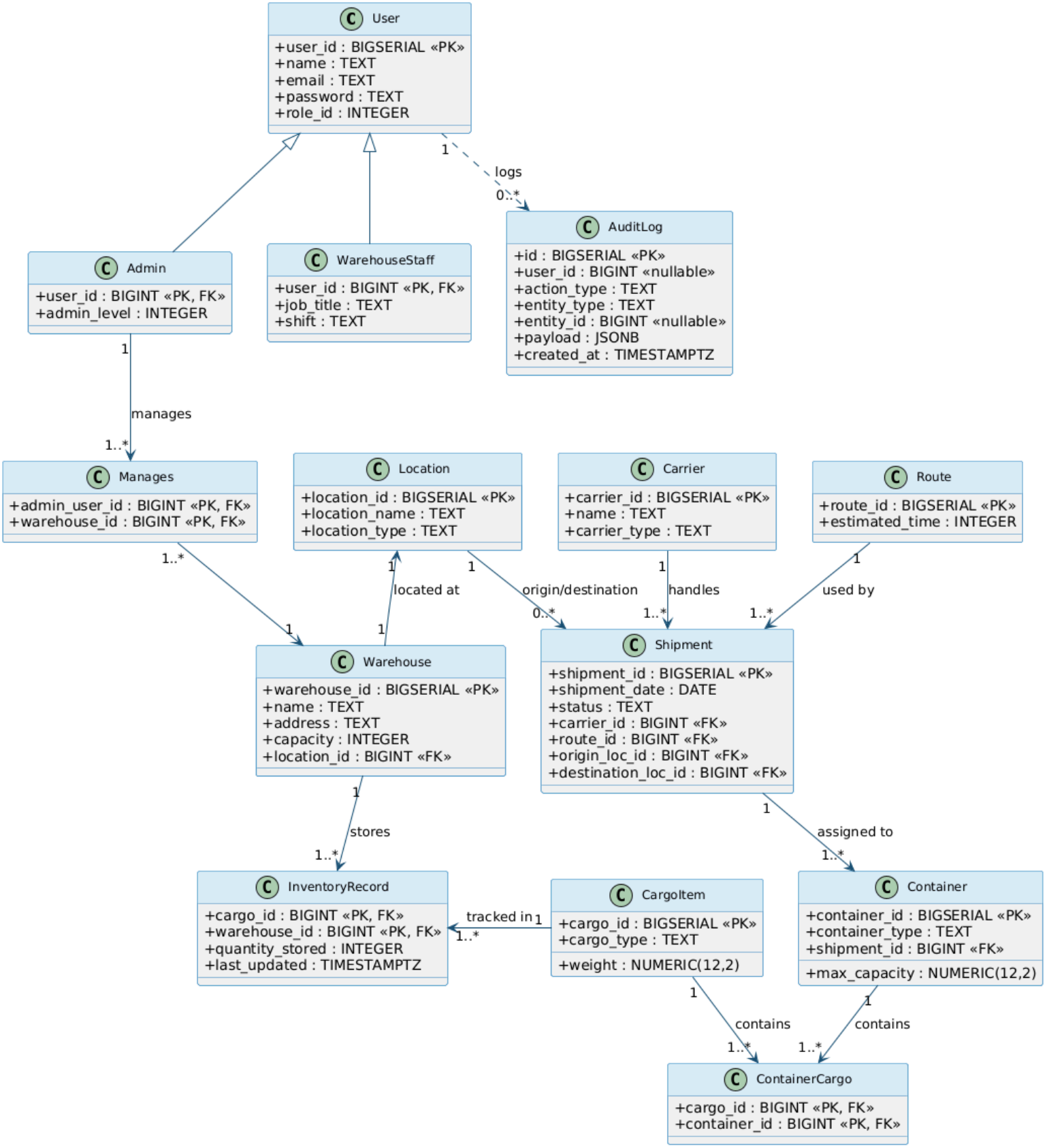
```sql
-- Insert a carrier and a route
INSERT INTO carriers(name, carrier_type)
VALUES ('AirFast', 'AIR');

INSERT INTO routes(estimated_time)
VALUES (2);

-- Insert a shipment
INSERT INTO shipments(shipment_date, status, carrier_id, route_id, origin_loc_id,
destination_loc_id)
VALUES (CURRENT_DATE, 'IN_TRANSIT', 1, 1, 1, 2);

-- Insert a container linked to the shipment
INSERT INTO containers(container_type, max_capacity, shipment_id)
VALUES ('40ft Dry', 28000, 1);

-- Insert a cargo item and seed its inventory in a warehouse
INSERT INTO cargo_items(cargo_type, weight)
VALUES ('Consumer Electronics', 12000);

INSERT INTO inventory_records(cargo_id, warehouse_id, quantity_stored)
VALUES (1, 1, 10);

-- Load cargo into a container
INSERT INTO container_cargo(cargo_id, container_id)
VALUES (1, 1);

-- Reassign a container to a different shipment
UPDATE containers
SET shipment_id = 1
WHERE container_id = 2;

-- Change shipment status to delivered
UPDATE shipments
SET status = 'DELIVERED'
WHERE shipment_id = 1;

-- Update inventory quantity after a staff adjustment
UPDATE inventory_records
SET quantity_stored = quantity_stored + 5,
    last_updated = NOW()
WHERE cargo_id = 1 AND warehouse_id = 1;

-- Delete a carrier (realistic: insert then remove a test carrier)
INSERT INTO carriers(name, carrier_type)
VALUES ('GroundMove', 'LAND');
```

```
DELETE FROM carriers
WHERE name = 'GroundMove' AND carrier_type = 'LAND';

-- Select all shipments filtered by status, ordered by date
SELECT *
FROM shipments
WHERE status = 'IN_TRANSIT'
ORDER BY shipment_date DESC;

-- Select inventory records for a specific warehouse, ordered by quantity
SELECT ir.cargo_id, ci.cargo_type, ci.weight, ir.quantity_stored, ir.last_updated
FROM inventory_records ir
JOIN cargo_items ci ON ci.cargo_id = ir.cargo_id
WHERE ir.warehouse_id = 1
ORDER BY ir.quantity_stored DESC;

-- Select all containers assigned to a given shipment
SELECT c.container_id, c.container_type, c.max_capacity
FROM containers c
WHERE c.shipment_id = 1;

-- Select all staff members working the day shift
SELECT u.name, u.email, ws.job_title, ws.shift
FROM users u
JOIN warehouse_staff ws ON ws.user_id = u.user_id
WHERE ws.shift = 'DAY'
ORDER BY u.name;

-- Select using views
SELECT *
FROM view_shipment_summary
ORDER BY shipment_id;

SELECT *
FROM view_warehouse_inventory
ORDER BY warehouse_id;
```

Table VIEWS:

As you can see, we use views for some of the select functions, just for ease of access for displaying data on the frontend. Here is the code for the creation of the views as well:

```
CREATE OR REPLACE VIEW view_shipment_summary AS
SELECT
  s.shipment_id,
  s.status,
  s.shipment_date,
```

```
  c.name AS carrier_name,
  r.route_id,
  s.origin_loc_id,
  s.destination_loc_id,
  COUNT(DISTINCT ct.container_id) AS container_count,
  COUNT(DISTINCT cc.cargo_id) AS cargo_item_count
FROM shipments s
JOIN carriers c ON c.carrier_id = s.carrier_id
JOIN routes r ON r.route_id = s.route_id
JOIN locations lo ON lo.location_id = s.origin_loc_id
JOIN locations ld ON ld.location_id = s.destination_loc_id
LEFT JOIN containers ct ON ct.shipment_id = s.shipment_id
LEFT JOIN container_cargo cc ON cc.container_id = ct.container_id
GROUP BY
  s.shipment_id,
  s.status,
  s.shipment_date,
  c.name,
  r.route_id,
  s.origin_loc_id,
  s.destination_loc_id;

CREATE OR REPLACE VIEW view_warehouse_inventory AS
SELECT
  w.warehouse_id,
  w.name AS warehouse_name,
  SUM(ir.quantity_stored) AS total_units,
  SUM(ir.quantity_stored * COALESCE(ci.weight, 0)) AS total_weight
FROM warehouses w
LEFT JOIN inventory_records ir ON ir.warehouse_id = w.warehouse_id
LEFT JOIN cargo_items ci ON ci.cargo_id = ir.cargo_id
GROUP BY w.warehouse_id, w.name;
```

UML Class Diagram: